

ENTERPRISE SALES ANALYTICS DATA PLATFORM (ESADP)

Sprint 0 — Project Initiation & Architecture

UC19: Building a Unified Sales Performance & Revenue Analytics Platform

Enterprise Data Engineering Capstone Project

Submitted by

Hitashri M (26MML1024)
Kusuma Sree (26MML1021)
Krishnaveni M (26MML1008)

Client: ABC Global Retail Ltd.

Methodology: Agile Scrum (3 Sprints)

Table of Contents

1. Project Charter 3

2. Business Requirement Document (BRD) 3

3. Solution Architecture 6

4. Git Repository Structure 7

5. Sprint Backlog (Product Backlog) 8

6. Process Overview — End to End..... 8

7. Expected Final Output 9

1. Project Charter

Field	Content
Project Name	Enterprise Sales Analytics Data Platform (ESADP)
Sponsor	VP of Sales & Revenue, ABC Global Retail Ltd. (fictional)
Project Goal	Integrate sales, customer, product, dealer, pricing, and inventory data from independent enterprise systems into a governed PostgreSQL warehouse with executive Power BI reporting
Duration	3 Sprints
Methodology	Agile Scrum
Team	Hitashri M (26MML1024), Kusuma Sree (26MML1021), Krishnaveni M (26MML1008)
Success Criteria	An end-to-end pipeline — raw multi-format sources to staging to warehouse to dashboard — with documented lineage and data quality
Real-World Data Basis	Olist Brazilian E-Commerce dataset (8 sources) + Olist Marketing Funnel dataset (1 source) + self-generated Inventory dataset anchored to real product IDs (1 source) — 10 integrated systems total

2. Business Requirement Document (BRD)

2.1 Business Context

ABC Global Retail Ltd. sells through retail stores, e-commerce, distributors, dealers, and corporate channels across multiple countries. Point of Sale, Sales Order Management, CRM, ERP, Product Catalog, Pricing, Inventory, Marketing, and Dealer Management systems all operate independently today, leaving no single view of sales performance, revenue trend, inventory position, or customer behaviour across the business.

2.2 Stakeholders

Stakeholder	Role	Pain Point / Need
VP, Sales & Revenue	Executive Sponsor	No unified view of sales performance across channels
Regional Sales Manager	Business User (Sales Order Mgmt)	Cannot compare regional performance without manually pulling reports from each system
CRM / Customer Insights Lead	Business User (CRM)	No consolidated view of customer purchase behavior
Merchandising / Product Lead	Business User (Product Catalog)	Cannot identify underperforming products or categories without manual joins
Dealer / Partner Manager	Business User (Dealer Mgmt)	No consolidated view of seller performance across the marketplace
Finance / Pricing Analyst	Business User (Pricing / ERP)	Cannot reconcile payment and pricing data against sales volume

Stakeholder	Role	Pain Point / Need
Inventory Planner	Business User (Inventory Mgmt)	No visibility into stock position relative to sales demand
Marketing Analyst	Business User (Marketing Campaign)	No unified view of lead-to-deal conversion performance
Data Engineering Team	Technical Owner	Needs a governed staging and warehouse layer to build reporting on
IT / Data Governance	Compliance	Needs lineage, audit logs, and a data dictionary for audit readiness

2.3 Business Objectives

- Consolidate sales order, customer, product, dealer, pricing, marketing, and inventory data into one warehouse.
- Provide sales leadership a single, trusted view of revenue and regional performance.
- Enable merchandising to evaluate product and category performance without manual joins.
- Enable the dealer and partner team to compare seller performance.
- Give inventory planning a data-backed view of stock position alongside sales trend.
- Enable marketing to track lead-to-deal conversion performance alongside sales outcomes.
- Provide governance so every number on the executive dashboard is traceable back to its source.

2.4 Source Inventory — Data Sources and Format Mapping

All ten enterprise systems named in the use case are represented using real, publicly available data. The table below maps each system to its underlying dataset and the file format it is preserved or converted into for ingestion.

#	Data Source	Dataset Used	Format Preserved As
1	Sales Order Management	olist_orders_dataset.csv	CSV (native)
2	Point of Sale (POS)	olist_order_items_dataset.csv	SQL (converted)
3	CRM	olist_customers_dataset.csv	JSON (converted)
4	E-Commerce Platform	Same transactional core as #1/#2, documented not re-ingested	JSON (converted)
5	ERP	olist_order_payments_dataset.csv	XML (converted)
6	Dealer Management	olist_sellers_dataset.csv	XML (converted)
7	Product Catalog	olist_products_dataset.csv + product_category_name_translation.csv	Excel (converted)
8	Pricing Management	price / freight_value extracted from olist_order_items_dataset.csv	Excel (converted)
9	Inventory Management	Self-generated (generate_inventory_data.py, anchored to real product_ids)	SQL (native)

#	Data Source	Dataset Used	Format Preserved As
10	Marketing Campaign	olist_marketing_qualified_leads_dataset.csv + olist_closed_deals_dataset.csv	CSV (native)

2.5 Data Dictionary

The data dictionary is the field-level reference that maps every column landing in the Bronze staging tables back to its source system, data type, and business meaning. It is compiled in full during Sprint 1 (PB-07) as each of the ten sources is ingested and staged. The excerpt below illustrates the structure of that dictionary using a representative sample of fields drawn from different source systems; the complete field-level dictionary covering all staged columns is maintained as a Sprint 1 governance artifact.

Source System	Field Name	Data Type	Description	Sample Value
Sales Order Management	order_id	VARCHAR	Unique identifier for a customer order	e481f51...9dc
Sales Order Management	order_purchase_timestamp	TIMESTAMP	Date and time the order was placed	2018-01-25 14:22:31
CRM	customer_id	VARCHAR	Unique identifier for a customer record	3f5c8a2...e17
Product Catalog	product_category_name	VARCHAR	Product category assigned by the seller	beleza_saude
Pricing Management	price	NUMERIC(10,2)	Unit price of the item at time of sale	89.90
Dealer Management	seller_id	VARCHAR	Unique identifier for a marketplace seller / dealer	3442f80...b23
Inventory Management	stock_quantity	INTEGER	Self-generated on-hand quantity anchored to product_id	145
Marketing Campaign	mql_id	VARCHAR	Unique identifier for a marketing-qualified lead	dc4802a...f6b

3. Solution Architecture

3.1 High-Level Architecture Diagram

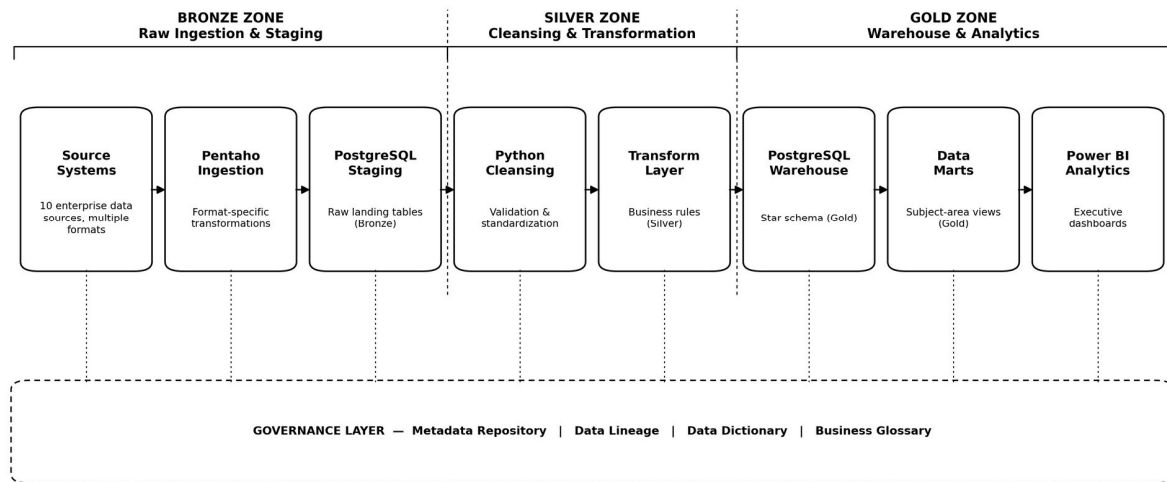


Figure 1. ESADP End-to-End Data Architecture (Medallion Design: Bronze – Silver – Gold)

The platform follows a medallion architecture. The Bronze zone ingests and stages raw data exactly as received from each of the ten source systems. The Silver zone cleanses, validates, and standardizes that data. The Gold zone holds the curated star-schema warehouse, subject-area data marts, and the executive analytics layer. A governance layer — metadata, lineage, data dictionary, and glossary — runs parallel to all three zones.

3.2 Role of PostgreSQL

PostgreSQL serves two distinct roles in this architecture. First, as the staging layer: a raw landing zone where each source system lands in its own staging table, structurally unmodified from source, keeping ingestion (Sprint 1) and cleansing (Sprint 2) as separate, auditable steps. Second, as the warehouse layer: the analytics-ready store, held in a separate schema of the same database, holding the star schema (fact and dimension tables) that Power BI connects to directly.

Using one relational engine for both staging and warehouse keeps the platform's scope appropriate for this project while still requiring genuine schema design, indexing, and SQL query-writing — the core skills this sprint is assessing. In short, PostgreSQL functions as both the inbox (staging) and the filing system (warehouse): everything upstream of it is format conversion, and everything downstream of it is reporting.

3.3 Layer Responsibilities

Layer	Tool	Responsibility
Source (Bronze)	10 real-world enterprise data sources	Real data standing in for ABC's enterprise systems
Ingestion (Bronze)	Pentaho (Spoon)	Format-specific extraction into staging; logging and exception handling
Staging (Bronze)	PostgreSQL	Raw landing zone, no transformation logic
Cleansing (Silver)	Python (Pandas)	Validation, standardization, data quality checks
Transformation (Silver)	Python / SQL	Business rules, deduplication, conformance

Layer	Tool	Responsibility
Warehouse (Gold)	PostgreSQL (Star Schema)	Curated, analytics-ready facts and dimensions
Data Marts (Gold)	PostgreSQL	Subject-area views for reporting
Analytics (Gold)	Power BI	Executive-facing dashboards
Governance	Metadata / Lineage / Markdown	Data dictionary, lineage, glossary — parallel to all layers

4. Git Repository Structure

```

ESADP/
├── docs/
├── config/
├── datasets/
│   ├── bronze/
│   │   ├── ingestion/
│   │   └── staging/
│   ├── silver/
│   │   ├── cleansing/
│   │   ├── validation/
│   │   └── transformations/
│   └── gold/
│       ├── warehouse/
│       ├── datamarts/
│       └── analytics/
├── pentaho/
│   ├── transformations/
│   └── jobs/
├── python/
├── sql/
├── metadata/
├── lineage/
├── dashboards/
├── deployment/
├── tests/
└── README.md

```

The repository will be initialised and maintained collaboratively by the team throughout the project, with all three members contributing commits ahead of each submission milestone.

5. Sprint Backlog (Product Backlog)

ID	User Story	Sprint
PB-01	Scaffold the Git repository using the medallion folder structure (bronze / silver / gold) so all members commit consistently	0
PB-02	Draft and finalize the Business Requirement Document to lock project scope before development starts	0
PB-03	Design the solution architecture, covering every medallion layer and the role of PostgreSQL	0
PB-04	Build the sprint backlog to plan and track work across all three sprints	0
PB-05	Export each of the 10 source systems into its designated format (CSV, Excel, JSON, XML, SQL)	1

ID	User Story	Sprint
PB-06	Build Pentaho transformations converting each format and loading it into PostgreSQL Bronze staging tables	1
PB-07	Compile a complete data dictionary covering every staged field	1
PB-08	Implement logging and exception handling on every ingestion pipeline	1
PB-09	Maintain incremental Git commit history showing ingestion progress	1

6. Process Overview — End to End

1. Real datasets (Olist e-commerce, Olist marketing funnel, and the self-generated inventory dataset) are identified and mapped to the ten enterprise systems defined in Section 2.4.
2. Each source is exported or converted into its designated format: CSV, Excel, JSON, XML, or SQL.
3. Pentaho transformations ingest each format and load the data, unmodified, into matching PostgreSQL Bronze staging tables.
4. Python scripts cleanse and validate the staged data for quality issues — duplicates, missing values, inconsistent categories — in the Silver zone.
5. A star schema is designed, and the PostgreSQL Gold warehouse and data marts are populated from the Silver-layer data.
6. Governance documentation — data dictionary, lineage, source-to-target mapping, glossary — is maintained alongside every layer.
7. Power BI connects to the Gold layer and builds executive dashboards.
8. The full pipeline is orchestrated through Pentaho Jobs, validated through reconciliation checks, and released through a tagged Git commit.

7. Expected Final Output

The project's deliverable is a working, documented data pipeline rather than a report alone. This includes: a populated PostgreSQL warehouse (star schema) queryable via SQL; a set of Pentaho transformation and job files reproducing the full ingestion and load process from raw source files; a Power BI dashboard connected to the warehouse, reporting on sales, revenue, customer, dealer, and inventory metrics; and a complete governance document set — data dictionary, source-to-target mapping, lineage diagram, and glossary — allowing any reviewer to trace a dashboard figure back to its original source file.

Every design decision made in later sprints should be traceable to a scope or architecture decision documented here in Sprint 0.